AI Policy in PYTHIA Software Development

Stephen Mrenna
Fermilab[1]

October 21, 2026

Responsible AI-assisted coding and
common training aspects for the HSF

[1]Based on P. Ilten, *Agentic AI for BSM Event Generation and Analysis* workshop at the LPC, 9 September 2026.

Setting the stage

- `oss-ai-policies.netlify.app` (K. Holterhoff, RedMonk) summarizes the policies of some major open-source projects, rating quality, copyright, and ethics concerns as low, medium, or high

ALL SURVEYED ORGANIZATIONS — SELECTED ENTRIES

ORGANIZATION	TYPE	STANCE	EXTERNAL PRS	QUALITY	COPYRIGHT	ETHICS	YEAR
CPython	Project	PERMISSIVE	ALLOWED	High	Low	Low	2025
Debian	Project	PROPOSAL REJECTED	PATCH/CLS	High	High	Medium	2024
Fedora	Project	PERMISSIVE	ALLOWED	High	High	Medium	2025
Firefox	Project	PERMISSIVE	PATCH/CLS	High	Medium	Low	2025
FreeCAD	Project	STRICT PERMISSIVE	ALLOWED	High	Medium	Low	2026
Homebrew	Project	PERMISSIVE	ALLOWED	High	Low	Low	2025
Kubernetes	Project	STRICT PERMISSIVE	ALLOWED	High	Low	Medium	2025
Linux Foundation	Foundation	PERMISSIVE	VARIES/N/A	Low	High	Low	2023
Linux Kernel	Project	PERMISSIVE	PATCH/CLS	High	High	Low	2025
Linux man-pages	Project	FULL BAN	PATCH/CLS	High	High	Medium	2025
LLVM	Project	PERMISSIVE	ALLOWED	High	Medium	Low	2025
Matplotlib	Project	STRICT PERMISSIVE	ALLOWED	High	Medium	Medium	2026
NumPy	Project	DISCUSSION IN PROGRESS	ALLOWED	Unknown	Unknown	Unknown	2025
scikit-learn	Project	PERMISSIVE	ALLOWED	High	Low	Low	2025
SciPy	Project	PERMISSIVE	ALLOWED	High	Low	Medium	2025

Not comprehensive: 49/88 are permissive.

Setting the stage

RedMonk's ratings are qualitative: a close reading of each policy's language, framing, and stated justifications.

- high: the concern is "explicitly named as a primary motivation for the policy, appears multiple times in the document, or drives the policy's core requirements"
- medium: "mentioned but not as the central driver, or is implied through requirements without being explicitly emphasized"
- low: "minimal or no explicit attention in the policy text"

- quality: e.g. policies that "discuss reviewer burden, require contributors to explain their code, or cite 'AI slop' "
- copyright: e.g. "copyright contamination", DCO sign-off
- ethics: "transparency, consent, environmental impact, or community values"

PYTHIA Background

- PYTHIA is a general purpose HEP Monte Carlo event generator
  - originated from the Lund theory group
  - actively developed for 44 years, beginning in 1982
  - primarily driven by Torbjörn Sjöstrand
  - in 1994 I joined, contributing SUSY code
  - in 2001 Peter Skands joined, contributing BSM and hadronization code
  - ...
  - currently 11 authors, with a spokesperson and a collaboration board
  - significant contributions from PhD students

- PYTHIA is infrastructure for many HEP experiments, and
- it is also the primary research of most PYTHIA authors

PYTHIA development

New code follows a relatively standard review procedure.

- a public `gitlab.com` repository for releases
- a semi-private `gitlab.cern.ch` repository for internal development
- general guidance in `CODINGSTYLE`

---

1. create merge request
2. check all CI pass (simple build, physics tests, interfaces, floating point exceptions, memory tests)
3. physics (and code) reviewed by another PYTHIA member
4. code (and physics) reviewed by a PYTHIA maintainer
5. code is merged into the main branch

PYTHIA development

The review procedure has two objectives, and the second one is about people, not code.

---

1. ensure that only bug-free, relevant, quality code is included into PYTHIA
2. build up expertise in the collaboration so that all parts of the code have a minimum of two members who understand it

---

Both objectives are critical for the sustainability of PYTHIA.

PYTHIA AI policy 1/5

Transparency.

> 1. All LLM usage by developers should be transparently reported.

- in practice, reported in the merge request description, the commit message, and code comments

PYTHIA AI policy 2/5

Human control.

> 2. LLMs should not have interactive access to non-forked Pythia8 code resources, e.g., git repositories. This means LLMs should not automatically create commits, merge requests, etc.

PYTHIA AI policy 3/5

Human responsibility.

> 3. Before any code is committed to non-forked Pythia8 code resources, all LLM generated code should be reviewed by the responsible developer.

PYTHIA AI policy 4/5

Maintainability and Sustainability.

> 4. All code must be fully understood by the developer providing the code. This means the developer should be able to explain to a reviewer any line of the proposed code.

Pythia AI policy 5/5

More subtle: LLM good at testing and poking the bear.

> 5. Review of code by LLMs can be a useful part of the code review process, but reviewers should be able to fully understand the code they are reviewing, independent of the LLM. If reviewers perform an LLM review, they must first perform an LLM independent review.

Where PYTHIA sits on RedMonk's Scale

My reading of the ratings

Organization	Stance	Quality	Copyright	Ethics
CPython	permissive	High	Low	Low
Linux Kernel	permissive	High	High	Low
PYTHIA	permissive	High	Low	Medium

Copyright is not addressed — an open question.

- high: the concern is "explicitly named as a primary motivation for the policy, appears multiple times in the document, or drives the policy's core requirements"
- medium: "mentioned but not as the central driver, or is implied through requirements without being explicitly emphasized"
- low: "minimal or no explicit attention in the policy text"

The policy in practice

A few experiences so far — in every case the LLM is one step in a chain, and a person has the last word.

1. LLM generated suggestions for speed-up $\rightarrow$ reworked by developer $\rightarrow$ code suggestions from reviewer $\rightarrow$ LLM review $\rightarrow$ code modifications from reviewer $\rightarrow$ approved by developer
2. new implementation of PDF set from developer $\rightarrow$ refactorization suggestion by reviewer $\rightarrow$ rewrite by LLM $\rightarrow$ rewrite by reviewer $\rightarrow$ review by LLM $\rightarrow$ code modifications from reviewer $\rightarrow$ approved by developer
3. updates to cross-section code from developer $\rightarrow$ physics review $\rightarrow$ technical review $\rightarrow$ LLM review $\rightarrow$ caught bug in old code $\rightarrow$ code modifications from reviewer $\rightarrow$ approved by developer
4. Me: why do we have any issues? Test the bug claim, write a demo, suggest a solution, and give me a patch < 30 lines – most are 3 or 4. Many claims are UFO sitings.

Some parting thoughts

Dilemma: We want policies now — but with thoughtful discussion.

- Now, because these tools allow us to concentrate on physics and that is where the funding is going (in the US at least) and it relates to our sustainability as a project
- Thoughtful, because we care about what we do and it's impact

- each policy depends on the project; the PYTHIA policy attempts to emphasize
  - trust
  - project sustainability
  - code quality

We should be cognizant of how policies shape student development, both in their learning and in their career, and how it shapes physics as a practice (understanding truths about Nature).